Elaborado por: Henry Antonio Mendoza Puerta

Ejercicio 02 — Academia Saber

Índice

Haga clic (en Word: Ctrl + clic) sobre un paso para ir a él. Al final de cada paso encontrará el enlace "↑ Volver al índice".

Paso 1. Analizar el enunciado

Paso 2. Crear el proyecto y los paquetes en NetBeans

Paso 3. Crear el modelo Estudiante (paquete com.usil.modelo)

Paso 4. Crear el service EstudianteService (paquete com.usil.service)

Paso 5. Crear el controlador EstudianteController (paquete com.usil.controlador)

Paso 6. Crear la vista VentanaEstudiantes (paquete com.usil.vista)

Paso 7. Crear el Main (paquete com.usil)

Paso 8. Ejecutar y probar con los datos del enunciado

¿Qué vamos a construir?

Una aplicación con **ventana Swing** que permite registrar estudiantes con sus tres notas, calcular su promedio y condición (aprobado o desaprobado), buscar por código y ver un resumen del salón. Se organiza con el patrón **MVC** y una capa **Service**:

- **Vista:** la ventana. Solo muestra datos y recibe clics. No tiene reglas de negocio.
- **Controlador:** intermediario. La vista le pide cosas; él las pide al service.
- **Service:** todas las reglas de negocio (validaciones, promedio, condición, búsquedas, resumen).
- **Modelo:** solo datos (la clase Estudiante).

Dependencia: Vista → Controlador → Service → Modelo. Cada capa solo conoce a la siguiente; el controlador **no** conoce la vista.

Clases que debe crear

Paquete	Clase	Rol
com.usil	Main	Crea el service, el controlador y la vista, y abre la ventana
com.usil.modelo	Estudiante	Única clase de modelo (solo datos)
com.usil.service	EstudianteService	Reglas de negocio, cálculos y búsquedas (con streams)
com.usil.controlador	EstudianteController	Recibe pedidos de la vista y llama al service
com.usil.vista	VentanaEstudiantes	Ventana Swing (JFrame Form diseñado en NetBeans)

Desarrollo paso a paso

Paso 1. Analizar el enunciado

Pregunta	Respuesta
¿Qué datos tiene un estudiante?	Código*, nombres*, nota 1*, nota 2*, nota 3*.
¿Qué reglas hay?	Código de 6 dígitos y no repetido; campos obligatorios; notas entre 0 y 20; promedio = (n1+n2+n3)/3; aprobado si el promedio es 11 o más.
¿Dónde van las reglas?	En la capa **Service**, nunca en la vista ni en el controlador.
¿Qué funciones tiene la ventana?	1) Registrar, 2) Ver la lista con promedio y condición, 3) Buscar por código, 4) Ver el resumen (promedio general, aprobados, desaprobados).
¿Cómo se muestran los errores?	En la ventana (cuadro de diálogo), nunca por consola.
¿Qué streams se exigen en el service?	Verificar si un código existe, buscar por código, calcular el promedio general y contar los aprobados.

↑ Volver al índice

Paso 2. Crear el proyecto y los paquetes en NetBeans

1. Menú **File → New Project → Java with Maven → Java Application**. Project Name: **Ejercicio02**. Group Id: **com.usil**. Finish.
2. Elimine la clase de ejemplo (App) que NetBeans crea.
3. Clic derecho sobre **Source Packages → New → Java Package** y cree: com.usil.modelo, com.usil.service, com.usil.controlador y com.usil.vista.
4. Cree la clase **Main** dentro de com.usil (la completaremos en el Paso 7).

↑ Volver al índice

Paso 3. Crear el modelo Estudiante (paquete com.usil.modelo)

Es la **única clase de modelo**: solo datos. Atributos privados, un constructor y getters. Ninguna regla de negocio.

Estudiante.java

```
package com.usil.modelo;

public class Estudiante {

    private final String codigo;
    private final String nombres;
    private final double nota1;
    private final double nota2;
    private final double nota3;

    public Estudiante(String codigo, String nombres, double nota1, double nota2, double
nota3) {
        this.codigo = codigo;
        this.nombres = nombres;
        this.nota1 = nota1;
        this.nota2 = nota2;
        this.nota3 = nota3;
    }

    public String getCodigo() { return codigo; }
    public String getNombres() { return nombres; }
    public double getNota1() { return nota1; }
    public double getNota2() { return nota2; }
    public double getNota3() { return nota3; }
```

```
}
```

Puntos clave del código

- Las notas se guardan como double ya validadas: el modelo recibe datos correctos.
- El promedio y la condición **no** se guardan aquí: se calculan en el service.
- Use Insert Code… → Constructor / Getter para generarlos rápido.

↑ Volver al índice

Paso 4. Crear el service EstudianteService (paquete com.usil.service)

Es el corazón del programa: aquí viven **todas las reglas de negocio**. No importa nada de javax.swing. Cuando una regla se incumple, lanza una IllegalArgumentException con el mensaje de error; la vista lo mostrará en un cuadro de diálogo.

EstudianteService.java

```
package com.usil.service;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Optional;
import com.usil.modelo.Estudiante;

public class EstudianteService {

    private static final double NOTA_MINIMA = 0;
    private static final double NOTA_MAXIMA = 20;
    private static final double NOTA_APROBATORIA = 11;

    private final List<Estudiante> estudiantes = new ArrayList<>();

    public void registrar(String codigo, String nombres, String nota1, String nota2,
String nota3) {
        if (esVacio(codigo) || esVacio(nombres) || esVacio(nota1) || esVacio(nota2) ||
esVacio(nota3)) {
            throw new IllegalArgumentException("Todos los campos son obligatorios.");
        }
        String codigoLimpio = codigo.trim();
        if (!codigoLimpio.matches("\\d{6}")) {
            throw new IllegalArgumentException("El código debe tener 6 dígitos.");
        }
        if (existeCodigo(codigoLimpio)) {
            throw new IllegalArgumentException("El código " + codigoLimpio + " ya está
registrado.");
        }
        double n1 = convertirNota(nota1);
        double n2 = convertirNota(nota2);
        double n3 = convertirNota(nota3);
        estudiantes.add(new Estudiante(codigoLimpio, nombres.trim(), n1, n2, n3));
    }

    public List<Estudiante> listar() {
        return Collections.unmodifiableList(estudiantes);
    }

    public Optional<Estudiante> buscarPorCodigo(String codigo) {
        return estudiantes.stream()
            .filter(e -> e.getCodigo().equals(codigo.trim()))
            .findFirst();
    }

    public double calcularPromedio(Estudiante estudiante) {
        return (estudiante.getNota1() + estudiante.getNota2() + estudiante.getNota3()) /
3;
    }
```

```
    public String obtenerCondicion(Estudiante estudiante) {
        return calcularPromedio(estudiante) >= NOTA_APROBATORIA ? "Aprobado" :
"Desaprobado";
    }

    public double calcularPromedioGeneral() {
        return estudiantes.stream()
            .mapToDouble(this::calcularPromedio)
            .average()
            .orElse(0);
    }

    public long contarAprobados() {
        return estudiantes.stream()
            .filter(e -> calcularPromedio(e) >= NOTA_APROBATORIA)
            .count();
    }

    public long contarDesaprobados() {
        return estudiantes.size() - contarAprobados();
    }

    private boolean existeCodigo(String codigo) {
        return estudiantes.stream().anyMatch(e -> e.getCodigo().equals(codigo));
    }

    private boolean esVacio(String texto) {
        return texto == null || texto.isBlank();
    }

    private double convertirNota(String texto) {
        double nota;
        try {
            nota = Double.parseDouble(texto.trim());
        } catch (NumberFormatException ex) {
            throw new IllegalArgumentException("Las notas deben ser numéricas.");
        }
        if (nota < NOTA_MINIMA || nota > NOTA_MAXIMA) {
            throw new IllegalArgumentException("Cada nota debe estar entre 0 y 20.");
        }
        return nota;
    }
}
```

Puntos clave del código

- registrar(...) recibe los datos como **texto** (tal cual los escribe el usuario) y los valida en orden: campos vacíos → código de 6 dígitos (matches("\\d{6}")) → código repetido → notas numéricas y entre 0 y 20.
- existeCodigo usa **anyMatch**: ¿algún estudiante tiene ese código?
- buscarPorCodigo usa **filter + findFirst** y devuelve un Optional (puede estar vacío si no existe).
- calcularPromedioGeneral usa **mapToDouble + average**: promedio de los promedios. orElse(0) evita error si no hay estudiantes.

- contarAprobados usa **filter + count**: cuenta los de promedio >= 11. Los desaprobados son el total menos los aprobados.
- convertirNota captura NumberFormatException para avisar “Las notas deben ser numéricas” en lugar de que el programa se caiga.
- listar() devuelve una lista no modificable para proteger los datos.

↑ Volver al índice

Paso 5. Crear el controlador EstudianteController (paquete com.usil.controlador)

Es el intermediario entre la vista y el service. Recibe los datos de la vista, llama al service y **devuelve resultados ya preparados** (por ejemplo, filas de texto para la tabla). **No conoce la vista**: no importa ninguna clase de javax.swing ni de com.usil.vista.

EstudianteController.java

```
package com.usil.controlador;

import com.usil.service.EstudianteService;
import java.util.List;
import java.util.Locale;
import java.util.Optional;
import java.util.stream.Collectors;

public class EstudianteController {

    private final EstudianteService service;

    public EstudianteController(EstudianteService service) {
        this.service = service;
    }

    public void registrarEstudiante(String codigo, String nombres, String nota1, String
nota2, String nota3) {
        service.registrar(codigo, nombres, nota1, nota2, nota3);
    }

    public List<String[]> listarEstudiantes() {
        return service.listar().stream()
            .map(e -> new String[]{
                e.getCodigo(),
                e.getNombres(),
                String.format(Locale.US, "%.2f", service.calcularPromedio(e)),
                service.obtenerCondicion(e)})
            .collect(Collectors.toList());
    }

    public Optional<String> buscarEstudiante(String codigo) {
        return service.buscarPorCodigo(codigo)
            .map(e -> String.format(Locale.US,
                "Código: %s%nNombres: %s%nNotas: %.1f - %.1f - %.1f%nPromedio:
%.2f%nCondición: %s",
                e.getCodigo(), e.getNombres(), e.getNota1(), e.getNota2(), e.getNota3(),
                service.calcularPromedio(e), service.obtenerCondicion(e)));
    }

    public String obtenerResumen() {
        return String.format(Locale.US,
            "Promedio general: %.2f%nAprobados: %d%nDesaprobados: %d",
            service.calcularPromedioGeneral(), service.contarAprobados(),
service.contarDesaprobados());
    }
}
```

Puntos clave del código

- Recibe el EstudianteService por el constructor (no lo crea): así se puede cambiar o probar fácilmente.
- registrarEstudiante no captura la excepción: la deja pasar para que la vista muestre el mensaje.
- listarEstudiantes usa un stream con una lambda para convertir cada Estudiante en una fila String[] (código, nombres, promedio, condición).
- buscarEstudiante devuelve un Optional<String>: el texto con los datos, o vacío si no existe.
- String.format(Locale.US, "%.2f", ...) muestra los decimales con punto y 2 decimales.

↑ Volver al índice

Paso 6. Crear la vista VentanaEstudiantes (paquete com.usil.vista)

Esta es la única clase que se **diseña visualmente** en NetBeans. Siga estos pasos:

1. Clic derecho sobre el paquete **com.usil.vista → New → JFrame Form…** (si no aparece, use Other → Swing GUI Forms → JFrame Form). Class Name: **VentanaEstudiantes**.
2. Se abre la pestaña **Design**. Clic derecho en el fondo del formulario → **Set Layout → Border Layout**.
3. Arrastre desde la **Palette** los componentes de la tabla siguiente y cambie su **nombre de variable** (clic derecho → *Change Variable Name…*) y su texto (*Properties → text*).

Variable	Componente	Dónde va	Propiedades
panelNorte	JPanel	Formulario, zona North	Layout: Border Layout
panelFormulario	JPanel	Dentro de panelNorte, Center	Layout: Grid Layout (5 filas, 2 columnas, gaps 5)
lblCodigo / txtCodigo	JLabel / JTextField	panelFormulario	text: “Código (6 dígitos):”
lblNombres / txtNombres	JLabel / JTextField	panelFormulario	text: “Nombres:”
lblNota1 / txtNota1	JLabel / JTextField	panelFormulario	text: “Nota 1:”
lblNota2 / txtNota2	JLabel / JTextField	panelFormulario	text: “Nota 2:”
lblNota3 / txtNota3	JLabel / JTextField	panelFormulario	text: “Nota 3:”
panelBotones	JPanel	Dentro de panelNorte, South	Layout: Flow Layout
btnRegistrar	JButton	panelBotones	text: “Registrar”
btnListar	JButton	panelBotones	text: “Ver lista”
btnBuscar	JButton	panelBotones	text: “Buscar por código”
btnResumen	JButton	panelBotones	text: “Ver resumen”
scrollTabla / tabla	JScrollPane / JTable	Formulario, zona Center	tabla → model: 4 columnas “Código, Nombres, Promedio, Condición”, 0 filas, sin edición
scrollDetalle / areaDetalle	JScrollPane / JTextArea	Formulario, zona South	areaDetalle: editable desmarcado, rows 5, columns 40

4. En las propiedades del JFrame: **title** = “Academia Saber - Registro de notas” y **defaultCloseOperation** = EXIT.

Importante: no haga doble clic sobre los botones del diseño; eso genera métodos vacíos innecesarios. Los eventos los asignaremos con lambdas en el constructor.

VentanaEstudiantes.java (parte que usted escribe)

```
package com.usil.vista;

import com.usil.controlador.EstudianteController;
import java.util.List;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

public class VentanaEstudiantes extends javax.swing.JFrame {

    private final EstudianteController controlador;

    public VentanaEstudiantes(EstudianteController controlador) {
        this.controlador = controlador;
        initComponents();
        setLocationRelativeTo(null);

        btnRegistrar.addActionListener(e -> registrar());
        btnListar.addActionListener(e -> mostrarLista());
        btnBuscar.addActionListener(e -> buscar());
        btnResumen.addActionListener(e -> mostrarResumen());
    }

    private void registrar() {
        try {
            controlador.registrarEstudiante(txtCodigo.getText(), txtNombres.getText(),
                txtNota1.getText(), txtNota2.getText(), txtNota3.getText());
            limpiarCampos();
            mostrarLista();
            mostrarMensaje("Estudiante registrado correctamente.");
        } catch (IllegalArgumentException ex) {
            mostrarError(ex.getMessage());
        }
    }

    private void mostrarLista() {
        List<String[]> filas = controlador.listarEstudiantes();
        DefaultTableModel modelo = (DefaultTableModel) tabla.getModel();
        modelo.setRowCount(0);
        filas.forEach(modelo::addRow);
    }

    private void buscar() {
        controlador.buscarEstudiante(txtCodigo.getText()).ifPresentOrElse(
            areaDetalle::setText,
            () -> mostrarError("No existe un estudiante con ese código."));
    }

    private void mostrarResumen() {
        areaDetalle.setText(controlador.obtenerResumen());
    }

    private void mostrarMensaje(String mensaje) {
        JOptionPane.showMessageDialog(this, mensaje, "Información",
JOptionPane.INFORMATION_MESSAGE);
    }

    private void mostrarError(String mensaje) {
        JOptionPane.showMessageDialog(this, mensaje, "Error", JOptionPane.ERROR_MESSAGE);
    }
```

```
    private void limpiarCampos() {
        txtCodigo.setText("");
        txtNombres.setText("");
        txtNota1.setText("");
        txtNota2.setText("");
        txtNota3.setText("");
    }

}
```

Puntos clave del código

- La vista recibe el **EstudianteController** por el constructor y lo guarda: la dependencia es Vista → Controlador.
- Después de initComponents() se asignan los eventos con **lambdas**: btnRegistrar.addActionListener(e -> registrar()).
- registrar() toma los textos de los campos, se los entrega al controlador y, si todo sale bien, limpia los campos, actualiza la tabla y avisa. Si el controlador lanza IllegalArgumentException, **muestra el mensaje en un cuadro de diálogo** (JOptionPane).
- mostrarLista() pide las filas al controlador y las carga en el modelo de la tabla (DefaultTableModel).
- buscar() usa el Optional: ifPresentOrElse muestra los datos en el área de texto o un mensaje de error si no existe.
- La vista **no valida nada** ni calcula promedios: solo muestra lo que el controlador le devuelve.

↑ Volver al índice

Paso 7. Crear el Main (paquete com.usil)

El Main arma las piezas **de adentro hacia afuera**: primero el service, luego el controlador (que recibe el service) y por último la vista (que recibe el controlador).

Main.java

```
package com.usil;

import com.usil.controlador.EstudianteController;
import com.usil.service.EstudianteService;
import com.usil.vista.VentanaEstudiantes;
import javax.swing.SwingUtilities;

public class Main {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            EstudianteService service = new EstudianteService();
            EstudianteController controlador = new EstudianteController(service);
            new VentanaEstudiantes(controlador).setVisible(true);
        });
    }
}
```

- SwingUtilities.invokeLater(...) es la forma correcta de abrir una ventana Swing.
- Observe el orden: new EstudianteService() → new EstudianteController(service) → new VentanaEstudiantes(controlador).

↑ Volver al índice

Paso 8. Ejecutar y probar con los datos del enunciado

Clic derecho sobre **Main.java → Run File**. Registre los estudiantes en este orden y verifique cada resultado:

Código	Nombres	Notas	Resultado esperado
202301	Ana Torres	14, 16, 15	Se registra. Promedio 15.00 — Aprobado
202302	Luis Paredes	8, 10, 12	Se registra. Promedio 10.00 — Desaprobado
202301	Pedro Ruiz	12, 12, 12	**Error** en ventana: el código 202301 ya está registrado
202305	Rosa León	22, 10, 10	**Error** en ventana: cada nota debe estar entre 0 y 20
202303	Marta Gil	18, 17, 19	Se registra. Promedio 18.00 — Aprobado
202304	Juan Pérez	10, 11, 12	Se registra. Promedio 11.00 — Aprobado

Luego pruebe los botones:

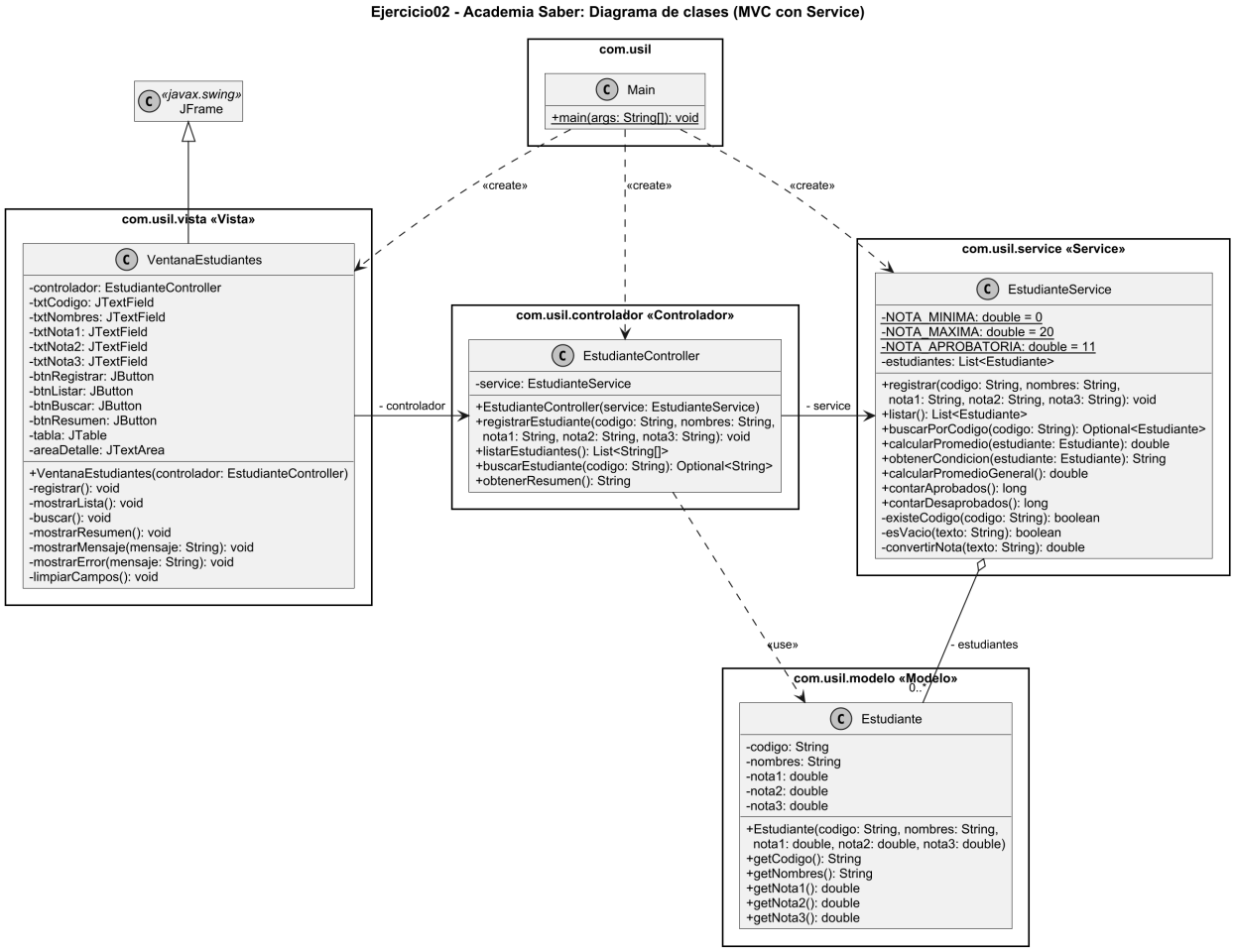
- **Ver lista:** la tabla muestra 4 estudiantes (202301, 202302, 202303, 202304) con su promedio y condición.
- **Buscar por código:** escriba 202303 y pulse el botón: se muestran los datos de Marta Gil (promedio 18.00, Aprobado). Con 999999 debe salir un mensaje de que no existe.
- **Ver resumen:** Promedio general: 13.50, Aprobados: 3, Desaprobados: 1.

Comprobación: (15 + 10 + 18 + 11) / 4 = 13.50.

↑ Volver al índice

Diagrama de Clase

Diagrama de clases de la solución



↑ Volver al índice